

Memory Management & Sorting

DSA · Intermediate · August cohort | Class notes — Stack DS · Call Stack · Stack vs Heap · Pass by Value · Sorting intro · 2 problems

How to read this. The wide left column is the class flow, in the order it was taught. The narrow right column is the margin — the things said out loud that are easy to lose: the traps, the reasons, and where each idea shows up in an interview. Every number, trace and code block in here was executed before printing.

► **NOTE** — clarification ★ **KEY** — the line to remember ⚠ **TRAP** — where people go wrong ♦ **INTERVIEW** — where it shows up

1 · Stack — the data structure

where the hour starts

Start with a stack of plates. You put a plate on the pile, you take a plate off the pile — and in both cases you only ever touch the plate on top. You never slide one out from the middle. That physical constraint is the data structure.

DEFINITION

A **stack** is a linear data structure in which **insertion and removal both happen at the same end — the top**.

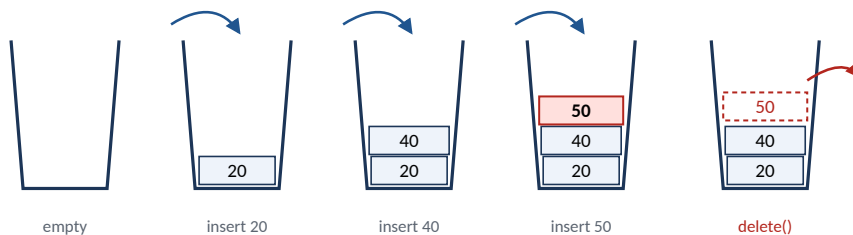
► NOTE

Everything else about a stack follows from that one rule. Don't memorise operations — re-derive them from "only the top is reachable".

► NAMES YOU'LL MEET

The insert is usually called `push`, the remove `pop`, and "look at the top without removing" is `peek` / `top`.

Trace it: three inserts, then one delete



20 goes in, then 40 sits on it, then 50 sits on 40. `delete()` can only take 50 — the last one that arrived.

★ LIFO — LAST IN, FIRST OUT

The element that entered most recently is the element that leaves first. And at any moment, **only the top element is accessible** — 20 and 40 are in there, but you cannot reach them without removing 50 first.

★ KEY

A stack does not let you search. If a problem needs "the most recent thing I saw", a stack is the answer. If it needs "any element", it isn't.

► COST

Because you only ever touch one end, push, pop and peek are all $O(1)$. No shifting, no scanning.

⚠ TRAP

"Only the top is accessible" is a rule of the *abstraction*, not of the memory. The other elements physically exist — the stack simply refuses to hand them to you.

2 · The Call Stack

a stack you have been using all along

Now the reason we opened with stacks. Every time your program calls a function, the machine uses a stack to keep track of where it is.

```
int add(int x, int y)    { return x + y; }
int product(int x, int y) { return x * y; }
int subtract(int x, int y) { return x - y; }

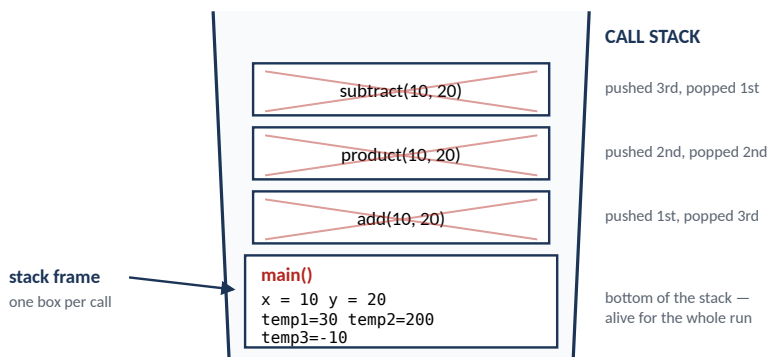
public static void main() {
    int x = 10;
    int y = 20;
    int temp1 = add(x, y);      // 30
    int temp2 = product(x, y);  // 200
    int temp3 = subtract(x, y); // -10
    System.out.println(temp1 + temp2 + temp3); // 220
}
```

► VERIFIED

Run as written this prints 220 — 30 + 200 + (-10).

► WATCH THE ORDER

`main` is itself a function, so it is the *first* frame pushed and the *last* one popped. It sits at the bottom of the whole picture.



★ KEY

A function's local variables live exactly as long as its frame does. That is why a local variable "disappears" when the function returns — the box was popped.

◆ INTERVIEW

This is the picture to draw when asked to trace recursion. Recursion is just many frames of the same function stacked on each other — and a recursion with no base case keeps pushing until the stack runs out of room (`StackOverflowError`).

► DO THIS

Re-draw the diagram for the moment *just after* `product` returns. Which boxes exist? What is written inside `main`'s box at that instant?

★ KEY

Each call gets a **stack frame** — a box holding that call's parameters and local variables. The frame is pushed when the call starts and popped the moment it returns. A crossed-out box above means that call has already finished and its frame is gone.

DEFINITION

1 • **Stack** — all **primitive data types** (int, float, double, char, boolean ...) and the **references to non-primitive data types** are stored here.

2 • **Heap** — the actual object that a reference points to is stored here. Arrays and objects are created in the heap.

So when you write `int[] ar = new int[3];` two different things happen in two different places: the *array* is built in the heap, and the *address of that array* is written into `ar`, which lives in the stack frame.

★ THE ONE-LINER

The stack holds the **name and the address**. The heap holds the **thing**.

► NOTE

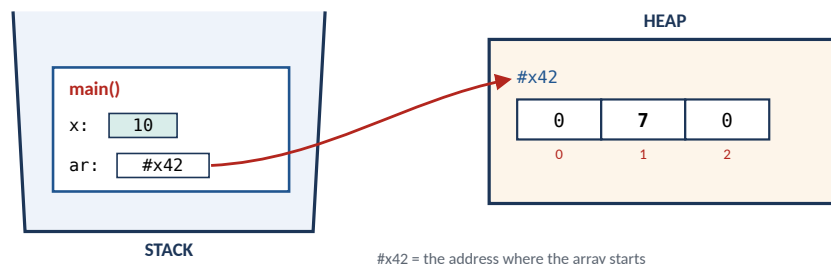
A reference is just a number — an address like `#x42`. The variable `ar` is a primitive-sized box holding that number; it is not the array.

► WHY TWO REGIONS

Stack frames are created and destroyed in a strict order, so the machine can free them automatically. Heap objects outlive the call that made them, so they need a garbage collector.

Example 1 — one reference, one array

```
public static void main() {
    int x = 10;
    int[] ar = new int[3];
    System.out.println(ar);    // [I@63947c6b ← an address
    System.out.println(ar[2]); // 0          ← zero-filled
    ar[1] = 7;
}
```



⚠ TRAP

`println(ar)` does **not** print the elements. It prints the reference, which is why you get something like `[I@63947c6b`. To see contents you need `Arrays.toString(ar)`.

► NOTE

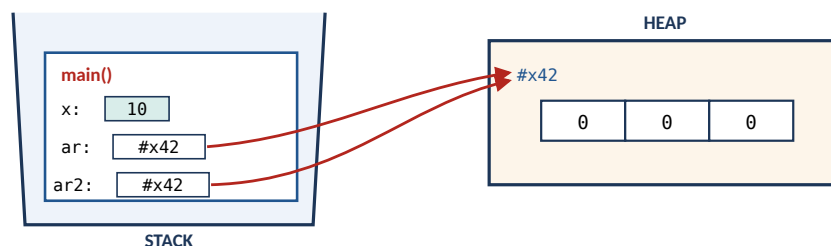
`ar[2]` is 0 even though nothing was assigned to it: a new `int[]` in the heap is zero-filled at creation.

► NOTE

`ar[1] = 7` changes the heap, not the stack. The number inside `ar` is untouched — it still says `#x42`.

Example 2 — two references, still one array

```
public static void main() {
    int x = 10;
    int[] ar = new int[3];
    int[] ar2 = ar;    // copies the ADDRESS
    System.out.println(ar);    // [I@2b193f2d
    System.out.println(ar2);   // [I@2b193f2d ← identical
}
```



★ KEY

Assignment between reference variables copies the arrow, never the thing at the end of the arrow.

◆ INTERVIEW

This is the "shallow copy vs deep copy" question. Shallow = copy the address (what happened here). Deep = allocate a fresh array in the heap and copy the values across.

⚠ THE CONSEQUENCE

`ar2 = ar` creates **no new array**. Two names, one object. Writing `ar2[0] = 99` means `ar[0]` is also 99 — there is nothing else it could mean.

► DO THIS

Add `ar2[0] = 99`; then print `ar[0]`. Predict before you run it.

Example 3 — re-pointing a reference, and the garbage collector

```
public static void main() {
    int[] ar = new int[3];
    System.out.println(ar);      // [I@355da254 (call it 5k)
    ar[1] = 9;
    ar[2] = 5;

    ar = new int[5];             // ar now points to a NEW array
    System.out.println(ar);      // [I@4dc63996 (7k)
}
```

► NOTE

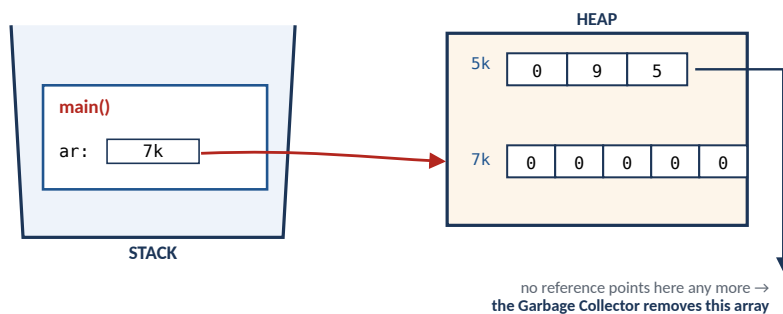
Re-assigning `ar` did not modify the old array. It overwrote the *address* stored in `ar`. The old array was abandoned, not changed.

⚠ TRAP

You cannot "get it back". Once the last reference to a heap object is overwritten, the object is unreachable from your code, permanently.

◆ INTERVIEW

"When does an object become eligible for garbage collection?" — the answer is this diagram: when no live reference reaches it.



★ KEY

The old 3-element array with 9 and 5 in it is still sitting in the heap — but nothing points to it any more, so it is **unreachable**. The garbage collector's job is exactly this: find heap objects that no reference reaches, and free them.

4 · In Java, parameters are passed by value

the sentence that explains everything below

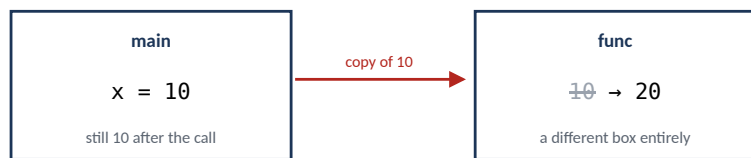
THE RULE — ONE SENTENCE, NO EXCEPTIONS

When you call a function, the value inside the variable is **copied** into the parameter. Two separate boxes, in two separate stack frames.

The rule never changes. What changes is *what the box contains* — a number, or an address. That single difference produces the two cases below.

Case 1 — primitive data type

```
main() {                                func(int x) {
    int x = 10;                          x = 20;
    func(x);                             }
    print(x);    // 10
}
```

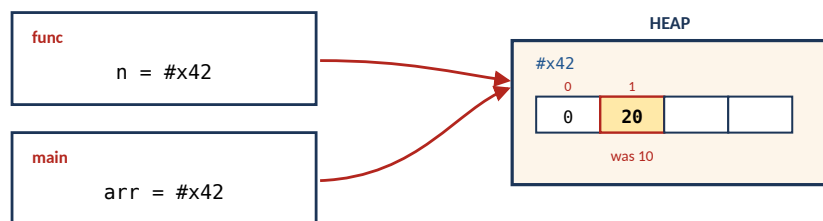


the frames are separate — `func` cannot reach `main`'s box

Output: 10. `func` changed its own copy and then its frame was popped. Nothing that happened in there survived.

Case 2 — non-primitive data type

```
main() {                                func(int[] n) {
    int arr[100];                       n[1] = 20;
    arr[1] = 10;                         }
    func(arr);
    print(arr[1]);    // 20
}
```



two separate boxes — but they hold the same address

Output: 20. The rule did not change — the address was copied. But both copies of the address lead to the *same* array in the heap, so a write through one is visible through the other.

★ KEY

Java is always pass-by-value. For objects, the *value being copied is the reference*. That is why it can look like pass-by-reference and behave like it — until you reassign.

► WHY

Both boxes are on the stack, in different frames. There is no path from `func`'s `x` back to `main`'s `x`.

► NOTE

Same name, different variable. The name `x` in the two frames has nothing to do with each other.

★ KEY

The copy is real. What is shared is not the variable — it is the object at the far end of the arrow.

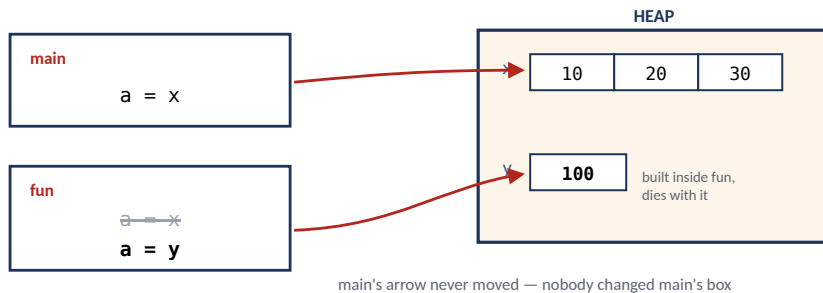
► THE TEST

Ask: did the function **write through** the arrow, or **replace** the arrow? Writing through is visible to the caller. Replacing is not.

Predict the output

```
function fun(a):
    a = new array[1]    // ← replaces the arrow
    a[0] = 100

function main():
    a = [10, 20, 30]
    fun(a)
    print(a[0])          // ?
```



★ ANSWER — 10

fun received a copy of the address of x. Then it **overwrote its own copy** with the address of a brand-new array y, and wrote 100 in there. Main's a was never touched — it still points at x, whose first element is still 10. When fun returns, y becomes unreachable and the garbage collector takes it.

⚠ THE WHOLE TRAP

Compare with Case 2. There the function wrote `n[1] = 20` — *through* the arrow. Here it wrote `a = new array[1]` — it *replaced* the arrow. The first is visible to main, the second can never be.

◆ INTERVIEW

This exact question is how interviewers check whether you actually understand pass-by-value, or just memorised "objects are passed by reference".

▶ DO THIS

Change `fun` to `a[0] = 100` (drop the `new`). What does main print now, and why?

5 · Sorting — introduction

new topic

DEFINITION

Sorting arranges the elements in **ascending** or **descending** order, based on **some factor**.

The word doing the work in that sentence is **factor**. In roughly 95% of cases the factor is simply the value itself — but it does not have to be. Here the same five numbers are sorted by their *number of factors*, and by that measure they are already in ascending order:

value	1	13	9	6	12
number of factors	1	2	3	4	6

$1 \rightarrow \{1\}$. $13 \rightarrow \{1, 13\}$. $9 \rightarrow \{1, 3, 9\}$. $6 \rightarrow \{1, 2, 3, 6\}$. $12 \rightarrow \{1, 2, 3, 4, 6, 12\}$.

★ KEY

Sorted means sorted **by something**. Name the factor before you sort, and half the "hard" sorting problems become one line.

◆ INTERVIEW

Custom factors are exactly what a comparator is for — sort intervals by start time, sort words by length, sort people by score then name.

Sorting libraries — use them

Language	Call	Time complexity
Java	<code>Arrays.sort()</code> / <code>Collections.sort()</code>	$O(n \log n)$
C++	<code>sort()</code>	$O(n \log n)$
Python	<code>arr.sort()</code>	$O(n \log n)$

★ KEY

You do not write the sort. You **call** the sort — and then you spend your thinking on *why* sorting makes the problem easy. Both problems below are exactly that shape.

► NOTE

Whenever a solution begins with a sort, the $O(n \log n)$ of the sort usually dominates the whole answer — quote it as the final complexity.

⚠ TRAP

`Arrays.sort(int[])` in Java sorts **ascending only**. For descending you need `Integer[]` with `Collections.reverseOrder()`, or sort ascending and walk the array backwards.

6 · Problem — Elements Removal

sorting, applied

PROBLEM

Given an array of n integers, minimise the **total cost** of emptying the array. The cost of removing one element is equal to the **sum of all the elements present at that moment** (the one being removed included).

Take `arr[] = [2, 8, 10]` and remove left to right, the obvious way:

Array right now	Remove	Cost paid
[2, 8, 10]	2	$2 + 8 + 10 = 20$
[8, 10]	8	$8 + 10 = 18$
[10]	10	10
Total		48

► READ IT CAREFULLY

The cost is the sum of what is *still there*. So every element you leave behind gets charged again on the next removal — and again, and again.

★ THE REFRAME

Don't ask "what should I remove first". Ask "**how many times will each element be charged?**" That question has a clean answer.

The insight

An element removed at step k (1-indexed) was present for all k removals up to that point, so it is **charged exactly k times**. You cannot change how many charges exist — $1\times, 2\times, 3\times \dots n\times$ are fixed. You can only choose *which element* gets which multiplier. So give the **biggest multiplier to the smallest element**: remove in **descending order**.

arr sorted descending

charged how many times

contributes

10	8	2
1	2	3
10	16	6

$10 + 16 + 6 = 32$ (against 48 for the naive order)

★ KEY

Big numbers must leave early so they are charged the fewest times. Small numbers can afford to hang around.

◆ INTERVIEW

This is the standard **exchange-argument greedy**: fixed multipliers, free assignment. Pair the largest multiplier with the smallest value. The same argument solves "minimum cost to connect ropes" and weighted-job-order questions.

► SANITY CHECK

Brute-forcing all 6 orderings of [2, 8, 10] gives 32 as the minimum. The greedy is not a guess.

Code

```
// sort descending, then charge arr[i] exactly (i+1) times
sort(arr, -1); // descending

int total_cost = 0;
for (int i = 0; i < n; i++) {
    total_cost = total_cost + (i + 1) * arr[i];
}
return total_cost;
```

Runnable Java, since `Arrays.sort` will not sort an `int[]` descending:

```
int n = arr.length;
Integer[] a = new Integer[n];
for (int i = 0; i < n; i++) a[i] = arr[i];
Arrays.sort(a, Collections.reverseOrder()); // descending

long totalCost = 0;
for (int i = 0; i < n; i++) totalCost += (long)(i + 1) * a[i];
return totalCost; // [2,8,10] → 32
```

Time complexity	$O(n \log n)$ — the sort dominates; the loop is $O(n)$
Space complexity	$O(1)$ extra beyond the sort ($O(n)$ if you box into <code>Integer[]</code>)

⚠ TRAP

`(i+1) * arr[i]` overflows `int` fast — n up to 10^5 with values up to 10^9 needs `long` for the running total.

► DO THIS

Run the formula on `[5, 5, 5]` by hand, then on `[1, 2, 3, 4]`. Check the second against removing them in the naive order.

7 · Problem — Noble Integers

sorting, applied again

PROBLEM

Given an array of **distinct** elements of size n , find the **count of noble integers**.
`arr[i]` is **noble** if the count of elements smaller than `arr[i]` is equal to `arr[i]`.

Example: `A[] = { 1, -5, 3, 5, -10, 4 }` → answer 3.

- 3 — elements smaller than 3 are 1, -5, -10 → three of them. ✓
- 5 — elements smaller than 5 are 1, 3, 4, -5, -10 → five of them. ✓
- 4 — elements smaller than 4 are 1, 3, -5, -10 → four of them. ✓

Sort, and the condition collapses

Sort ascending. Now for an array of distinct elements, the element sitting at index `i` has **exactly `i` elements smaller than it** — everything to its left, and nothing else.

	0	1	2	3	4	5
original	1	-5	3	5	-10	4
sorted	-10	-5	1	3	4	5
index i	0	1	2	3	4	5

★ THE WHOLE PROBLEM BECOMES ONE COMPARISON

`arr[i] == i`

Three hits — at $i = 3, 4$ and 5 . Answer 3. ✓

► BRUTE FORCE FIRST

For each element, scan the whole array and count smaller ones. Correct, and $O(n^2)$. Everything below is about killing that inner scan.

★ THE BRIDGE

"Number of elements smaller than me" is a **position**. Positions are what sorting hands you for free.

★ KEY

Index is the count of smaller elements — but only while every element is distinct. Hold that condition; the next section breaks it.

```

sort(arr);
int ans = 0;
for (int i = 0; i < n; i++) {
    if (arr[i] == i)
        ans++;
}
return ans; // TC: O(n log n)

```

► COMPLEXITY

$O(n \log n)$ for the sort + $O(n)$ for one pass = $O(n \log n)$. $O(1)$ extra space if you sort in place.

With duplicates — where `arr[i] == i` breaks

Drop the "distinct" guarantee and the rule fails immediately:

index	0	1	2	3	4
sorted	-10	1	1	3	100
elements actually smaller	0	1	1	3	4

The second 1 sits at index 2, but the number of elements smaller than it is still 1, not 2. It is noble, and `arr[i] == i` misses it. Duplicates do not push each other's count up — they share one count.

⚠️ SO THE FIX

Stop using `i`. Track the real count of smaller elements in a variable, and only update it when the value **changes**. Inside a run of equal values, the count is frozen at the index where the run started.

Dry run — [-3, 1, 1, 1, 5, 100]

index	0	1	2	3	4	5
value	-3	1	1	1	5	100
count (smaller than it)	0	1	1	1	4	5
noble?	-3 ≠ 0	✓	✓	✓	5 ≠ 4	100 ≠ 5

Answer: 3 — and note all three come from the same repeated value.

```

sort(arr);
int ans = 0;
int count = 0;

// index 0 has nothing to its left, so its count is 0
if (arr[0] == 0) ans++;

for (int i = 1; i < n; i++) {
    if (arr[i] != arr[i-1]) { // new value → run starts
        count = i;
        if (arr[i] == i) ans++;
    } else { // repeat → reuse the count
        if (arr[i] == count) ans++;
    }
}
return ans;

```

► WHY THE `ARR[0]` CHECK

The loop starts at `i = 1` because it compares against `arr[i-1]`. Index 0 has no left neighbour, so it is handled once, up front: it is noble exactly when `arr[0] == 0`.

★ KEY

Two branches, one idea: **new value** → **recompute the count**. **Repeat** → **reuse it**. That single line is the entire difference from the distinct version.

◆ INTERVIEW

"Handle duplicates" is the standard follow-up after you solve the distinct version. Expect it, and have this two-branch pattern ready — it reappears in every sorted-array-with-duplicates problem.

► VERIFIED

This code was run against a brute-force $O(n^2)$ checker on 3000 random arrays — every answer matched.

Time complexity	$O(n \log n)$ — sort, then a single pass
Space complexity	$O(1)$ extra

TRY IT YOURSELF — BEFORE THE NEXT CLASS

Run the final code by hand on the array we drew on the board:

index	0	1	2	3	4	5	6	7
	2	2	5	6	6	6	8	8

Write down count and ans after every iteration. Then do the same for $[-10, 1, 1, 3, 100]$ and check you get 3.

► HINT

Fill in the "elements smaller than me" row first, for all eight positions. Then compare it against the values. The answer for this one is not 3.

8 · One-page recall

Idea	The line to remember
Stack DS	Insert and remove happen at the same end. Only the top is accessible. LIFO . All operations $O(1)$.
Call stack	One stack frame per function call, holding its parameters and locals. Pushed on call, popped on return — which is why locals die when the function ends.
Stack vs Heap	Stack: primitives, and references to non-primitives. Heap: the actual arrays and objects. The stack holds the address; the heap holds the thing.
Aliasing	<code>ar2 = ar</code> copies the arrow, not the array. Two names, one object.
Garbage collection	Re-point the last reference and the old object becomes unreachable. The GC frees exactly what no reference reaches.
Pass by value	Always a copy. For objects the copy is the address . Writing <i>through</i> the arrow is visible to the caller; replacing the arrow is not.
Sorting	Ascending or descending by some factor . Library sorts are $O(n \log n)$ — call them, and spend your thinking on why sorting helps.
Elements Removal	The multipliers $1 \dots n$ are fixed; only the pairing is yours. Sort descending, answer = $\sum (i+1) \cdot \text{arr}[i]$. $O(n \log n)$.
Noble Integers	Sort, and "count of smaller elements" becomes the index. Distinct $\rightarrow \text{arr}[i] == i$. Duplicates $\rightarrow$ freeze count at the start of each run of equal values. $O(n \log n)$.

Verification. Every trace, table and answer in these notes was computed and checked before printing: 45 automated numeric checks (all passed), the Java snippets in sections 2–4 compiled and run with the outputs shown, and both problem solutions cross-checked against brute force — Noble Integers on 3000 random arrays, Elements Removal against every permutation of the example. Source: `memory_management_sorting_checks.py` and `Notes.java / Problems.java`.